

事务与数据库恢复技术

1.事务的基本概念

1.1 定义

事务是用户定义的一个数据库操作序列，这些操作必须全部一起执行，是一个不可分割的工作单位。

- 事务和程序不同：一个事务是一条SQL语句、一组SQL语句或者整个程序；而程序通常包含多个事务
- 事务是恢复和并发控制的基本单位

为什么要引入"事务"概念?

为了确保实现数据库的数据完整性

在一个数据库系统中,如果所有数据都满足数据完整性约束的定义,那么就称数据库处于一个一致性状态

影响数据完整性的因素:

- 并发
- 中止
- 崩溃

1.2 定义事务

显示定义方式:

BEGIN TRANSACTION SQL 语句1 SQL 语句2 COMMIT	BEGIN TRANSACTION SQL 语句1 SQL 语句2 ROLLBACK
➤ 事务正常结束 ➤ 提交事务的所有操作（读+更新） ➤ 事务中所有对数据库的更新写回到磁盘上的物理数据库中	➤ 事务异常终止 ➤ 事务运行的过程中发生了障，不能继续执行 ➤ 系统将事务中对数据库的所有已完成的操作全部撤销 ➤ 事务滚回到开始时的状态

隐式:

数据库管理系统按缺省规定自动划分事务

1.3 事务结束

□ COMMIT

- 事务正常结束
- 提交事务的所有操作（读+更新）
- 事务中所有对数据库的更新写回到磁盘上的物理数据库中

□ ROLLBACK

- 事务异常终止
- 事务运行的过程中发生了故障，不能继续执行
- 系统将事务中对数据库的所有已完成的操作全部撤销
- 事务滚回到开始时的状态

1.4 有关事务的SQL语句

- ‘事务’除了由一组对于数据库的访问操作构成以外，通常还应该包括少量的事务控制语句，用于定义一个事务的开始和结束(包括正常结束和非正常结束)。因此，与事务有关的控制语句主要有三条：

- 事务的开始 (begin transaction)
- 事务的结束
 - ① 正常结束
 - 提交事务 (commit transaction)
 - ② 非正常结束
 - 回退事务 (rollback transaction)

□ Begin transaction

- 现有的关系数据库管理系统并没有提供一个用于定义从什么时候开始一个事务的控制语句，事务的启动是隐式的
- 可以通过三种方式来启动一个新的事务：
 - ① 数据定义命令(DDL)
 - ② 将系统设为自动提交方式（打开自动提交标志）
 - ③ 数据操纵命令(DML)

❑ 事务的启动方式

① 数据定义命令(DDL)

- 每一条数据定义命令都将被作为一个单独的事务来执行
- 在此之前，用户的当前active事务将被自动提交

② 将系统设为自动提交方式（打开自动提交标志）

- 每一条数据库访问命令都将被作为一个单独的事务来执行，并根据执行结果由DBMS自动完成提交或回退

③ 数据操纵命令(DML)

- 当DBMS接收到来自某个用户的数据操纵请求时，首先检查该用户当前有没有正在运行的active事务：
- 如果有，该请求被作为当前active事务的组成部分去执行；
- 如果没有，DBMS将自动为该用户启动一个新的事务，该请求就被作为新事务的第一条数据访问请求去执行。

❑ Rollback transaction

- 取消在该事务执行过程中的所有操作结果，将数据库回滚至该事务开始前的状态，以便重新执行或放弃（abort）该事务
- 事务的执行过程不可能被‘回滚’，执行rollback操作是为了撤销该事务在之前的执行过程中对数据库中的数据所作的修改

❑ 保存点 (savepoint)

- 在事务的执行过程中，用户可以为该事务设置若干个保存点
- 用户事务可以使用 Rollback 命令将当前事务回退到前面的某个保存点sp，放弃“在保存点sp之后，回退操作之前”执行过的对数据库的所有访问操作，并继续执行当前事务
- 不带保存点的回退操作将结束并放弃整个事务

❑ 除了上述三条事务控制语句外，数据库管理系统通常还会提供下述几条与事务有关的控制命令(DCL)

① 设置事务的自动提交命令

SET AUTOCOMMIT ON | OFF ;

② 设置事务的类型

SET TRANSACTION READONLY | READWRITE ;

③ 设置事务的隔离级别

SET TRANSACTION ISOLATION LEVEL
READUNCOMMITTED |
READCOMMITTED |
READREPEATABLE |
SERIALIZABLE ;

❑ 用户可以使用这三条命令来设置自己事务的运行方式！

- ① **READUNCOMMITTED**：未提交读
 - 在该方式下，当前事务不需要申请任何类型的封锁，因而可能会‘读’到其他并发事务未提交的修改结果
 - 禁止一个事务以该方式去执行对数据的‘写’操作，以避免与其它并发事务的‘写’冲突现象
 - ② **READCOMMITTED**：提交读
 - 在‘读’一个数据对象A之前，需要先申请并获得数据对象A上的‘共享性’封锁，在‘读’操作执行结束之后立即释放该封锁
 - 以避免读取到其它并发事务未提交的修改结果
 - ③ **READREPEATABLE**：可重复读
 - 在‘读’数据对象A之前需要先申请并获得数据对象A上的‘共享性’封锁，并将该封锁维持到当前事务的结束
 - 可以避免其它的并发事务对当前事务正在使用的数据对象的修改
 - ④ **SERIALIZABLE**：可序列化(可串行化)
 - 并发事务以一种可串行化的调度策略实现其并发执行，以避免它们相互之间的干扰现象
- ❑ 不管采用何种隔离级别，在一个事务以‘写’方式访问数据对象A之前，需要先申请并获得数据对象A上的‘排它性’封锁，并将该封锁维持到当前事务的结束。

	Read 操作		Write 操作	
	锁类型	封锁时间	锁类型	封锁时间
READUNCOMMITTED 未提交读	No Lock	—	不允许执行 Write 操作	
READCOMMITTED 提交读	共享锁	读操作	排它锁	事务
READREPEATABLE 可重复读	共享锁	事务		
SERIALIZABLE 可串行化	共享锁	事务		

事务的隔离级别与封锁策略之间的关系

1.5 事务的特性(ACID)

- 原子性:

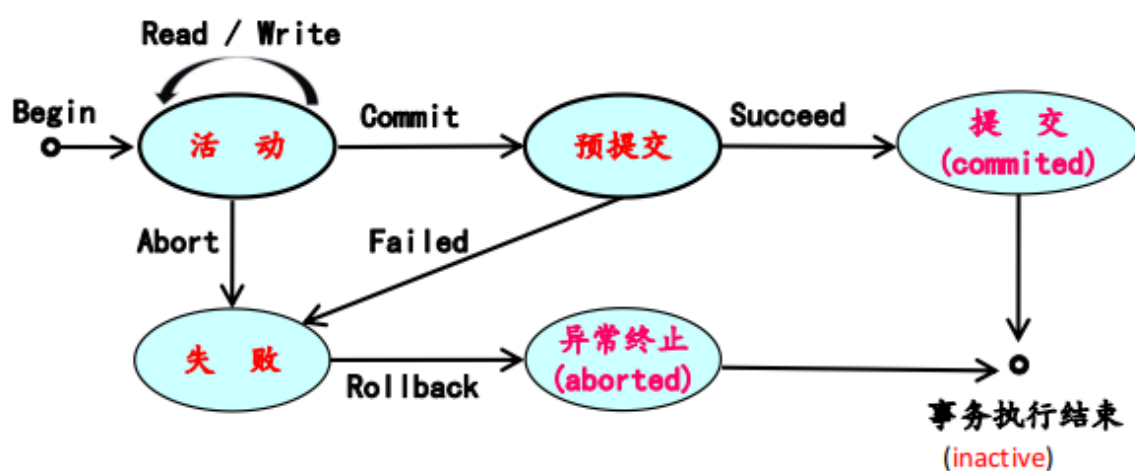
事务是数据库的逻辑工作单位,事务中包括的操作要么都做,要么都不做
- 一致性:
 - 事务执行的结果必须是使数据库从一个一致性状态变到另一个一致性状态
 - 一致性状态:数据库中只包含成功事务提交的结果
 - 不一致状态:数据库系统运行中发生故障,有些事务尚未完成就被迫中断
- 隔离性:
 - 一个事务的执行不能被其他的事务干扰
 - 一个事务内部的操作及使用的数据对其他并发事务是隔离的
 - 并发执行的各个事务之间不能互相干扰
- 持续性:
 - 一个事务一旦提交,它对数据库中数据的改变就应该是永久性的
 - 接下来的其他操作或故障不应该对其执行结果有任何影响

保证事务ACID特性是事务处理的任务

破坏事务ACID特性的因素:

- 多个事务并行运行时,不同事务的操作交叉执行
 - 数据库管理系统必须保证多个事务的交叉运行不影响这些事务的隔离性
- 事务在运行过程中被强行停止
 - 数据库管理系统必须被强行终止的事务对数据库和其他事务没有任何影响

1.6 事务活动



事务的状态变迁图

2.数据库恢复概述

3.故障的种类

4.恢复的实现技术

5.恢复策略

6.具有检查点的恢复技术

7.数据库镜像